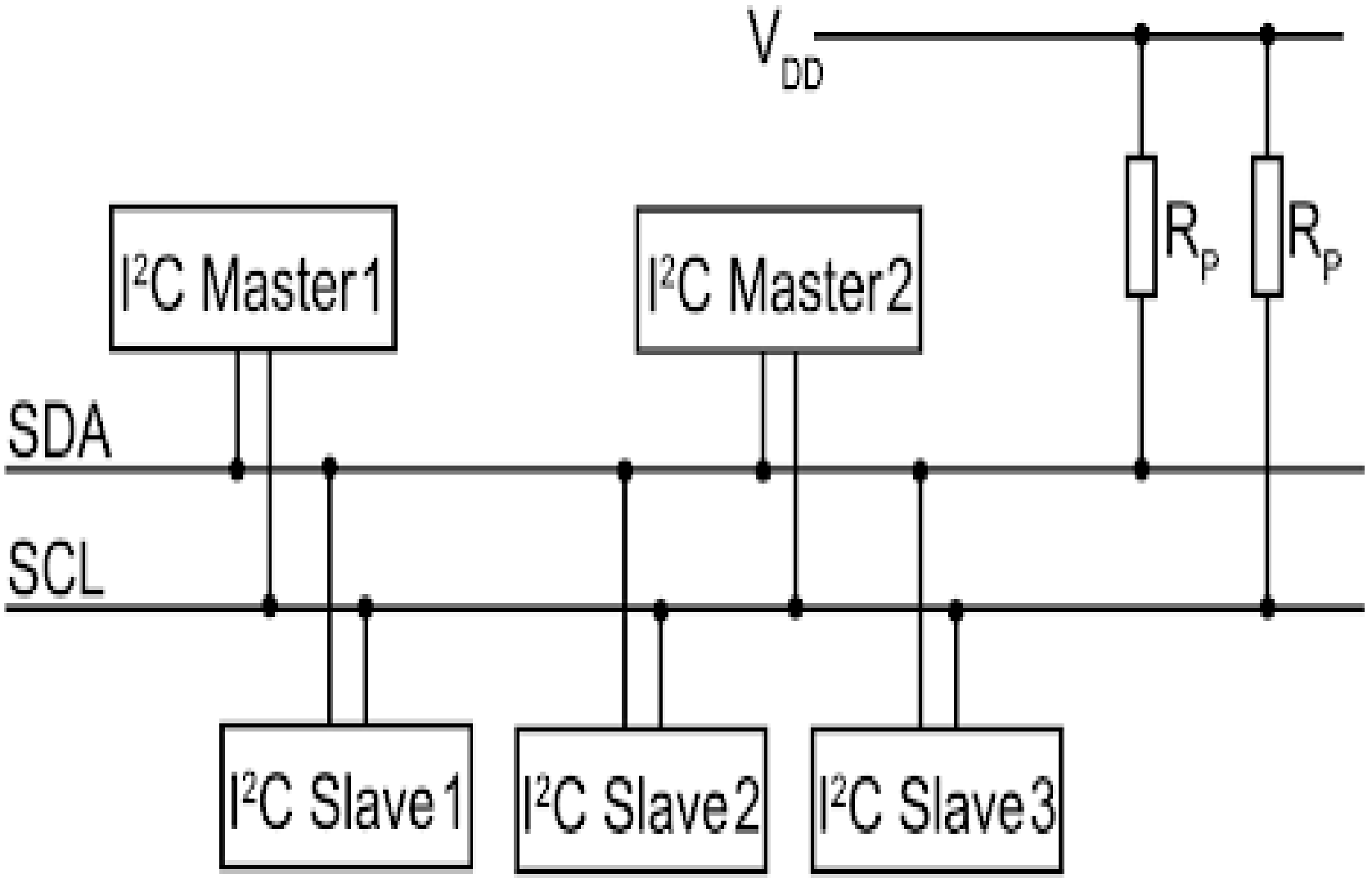
# I2C Bus


$V_{DD}$
$R_P$
$R_P$
I²C Master1
I²C Master2
SDA
SCL
I²C Slave1
I²C Slave2
I²C Slave3

# The GIST(Overview) of I2C Communication

Involving

One Master,One/More Slaves

AND

Two/More Masters,One/More Slaves

# I2C BUS EVENTS

- Idle Status of Bus
- Start Condition/Event
- Stop Condition/Event
- Repeated Start Condition/Event
- Data Validity & Transition For
  - Master Transmitting a Byte to a Slave.
  - Master Receiving aByte from a Slave.
- ACKNOWLEGMENT EVENTS
  - Master getting acknowledgement from a slave(ACK).
  - Master issuing Not Of Acknowledgement(NACK) onto bus for slave.
  - No Acknowledge Conditions.
  - Master giving acknowledgement to a slave(MACK).
- Clock Stretching done by slave
- Bus Arbitration done by master

# Notes

- Combinations of the braced events above may be used to implement one or more i2c communication algorithms depending on specific slave.
- Every communication algorithm will DIFINITELY

➢ begin with start condition

➢ end with stop condition.

➢ Involve Master Transmitting at least a Byte to a Slave.

➢ Involve Master getting atleast an acknowledgement from a slave(ACK).

```
 /* i2c_defines.h */
#ifndef __I2C_DEFINES_H__
#define __I2C_DEFINES_H__

//pins connected to i2c bus lines
#define SCL  2  //p0.2
#define SDA 3  //p0.3

//defines for pin function selection
#define SCL_EN  0x00000010
#define SDA_EN  0x00000040

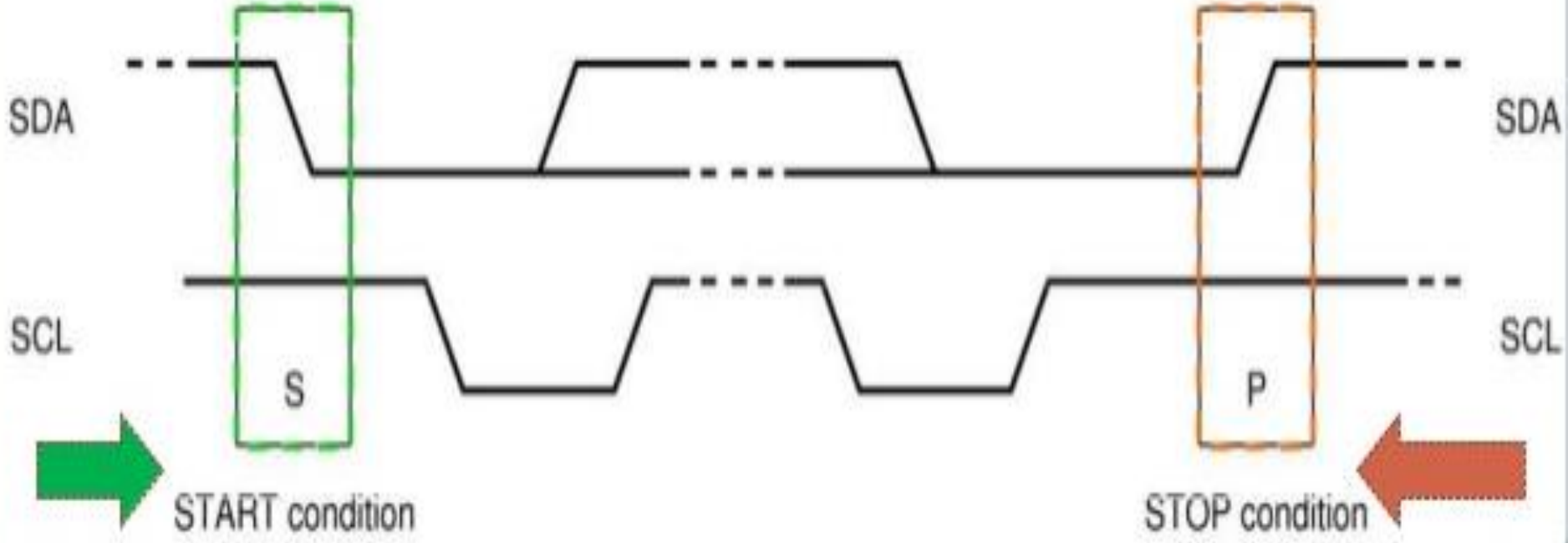
//defines for I2C_SPEED Configuration
#define CCLK            60000000  //Hz
#define PCLK            CCLK/4    //Hz
#define I2C_SPEED   100000    //Hz
#define LOADVAL     ((PCLK/I2C_SPEED)/2)

//bit defines for I2CONSET sfr
#define   SI_BIT 3
#define  STO_BIT 4
#define  STA_BIT 5
#define I2EN_BIT 6

#endif
```

# ➔Start & Stop Condition

- Start Condition – A high to low transition on SDA when SCL is high.
- Stop Condition – A low to high transition on SDA when SCL is high.



```
void i2c_start(void)
{
// start condition
I2CONSET=1<<STA_BIT;
//wait for SI bit status
while(((I2CONSET>>SI_BIT)&1)==0);
// clear start condition
I2CONCLR=1<<STA_BIT;
}
```

```
void i2cStart(void)
{
  SSETBIT(IOSET0,SDA);
  SSETBIT(IOSET0,SCL);
  SETBIT(IODIR0,SDA);
  SETBIT(IODIR0,SCL);
  SSETBIT(IOCLR0,SDA);
}
```

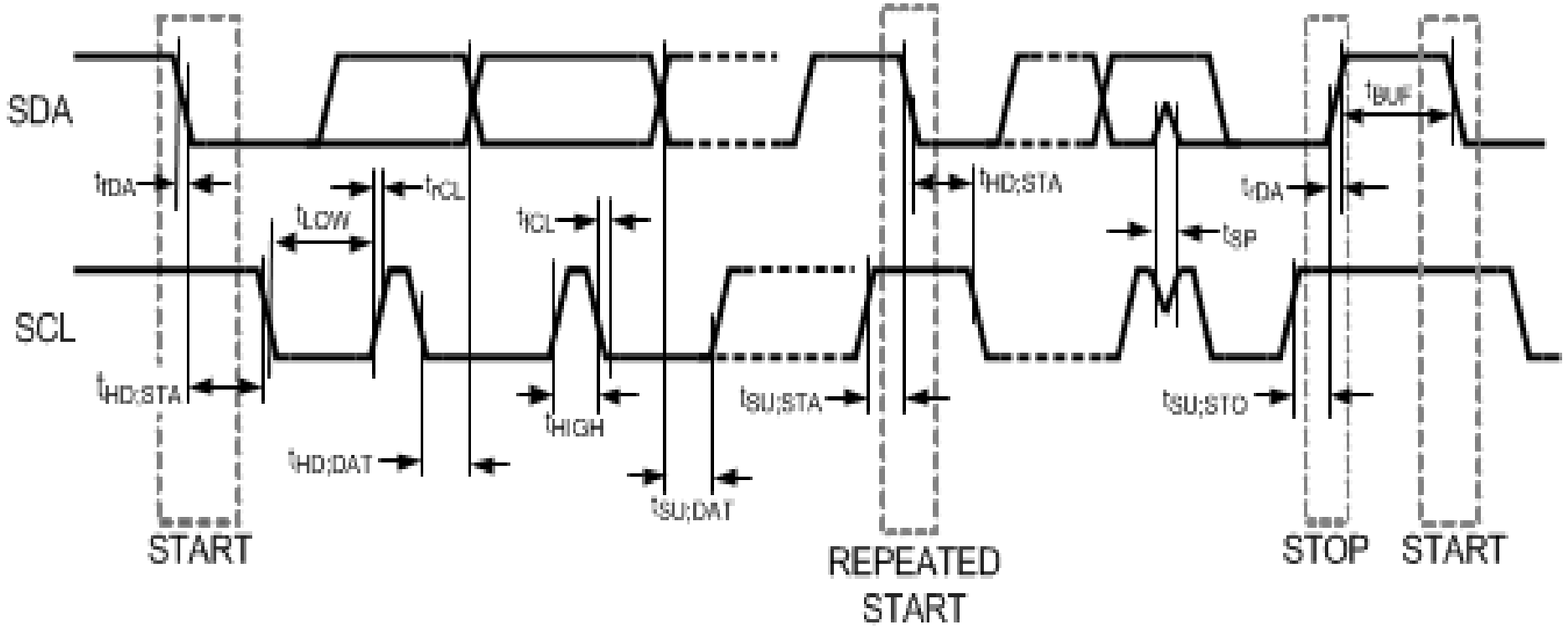
```
void i2c_stop(void)
{
   // issue stop condition
    I2CONSET=1<<STO_BIT;
      // clr SI bit status
   I2CONCLR = 1<<SI_BIT;
  //stop will cleared automatically

//while(((I2CONSET>>STO_BIT)&1));
}
```

```
void i2cStop(void)
{
    SCLRBIT(IOCLR0,SCL);
    SCLRBIT(IOCLR0,SDA);
    SSETBIT(IOSET0,SCL);
    SSETBIT(IOSET0,SDA);

}
```

# Repeated Start



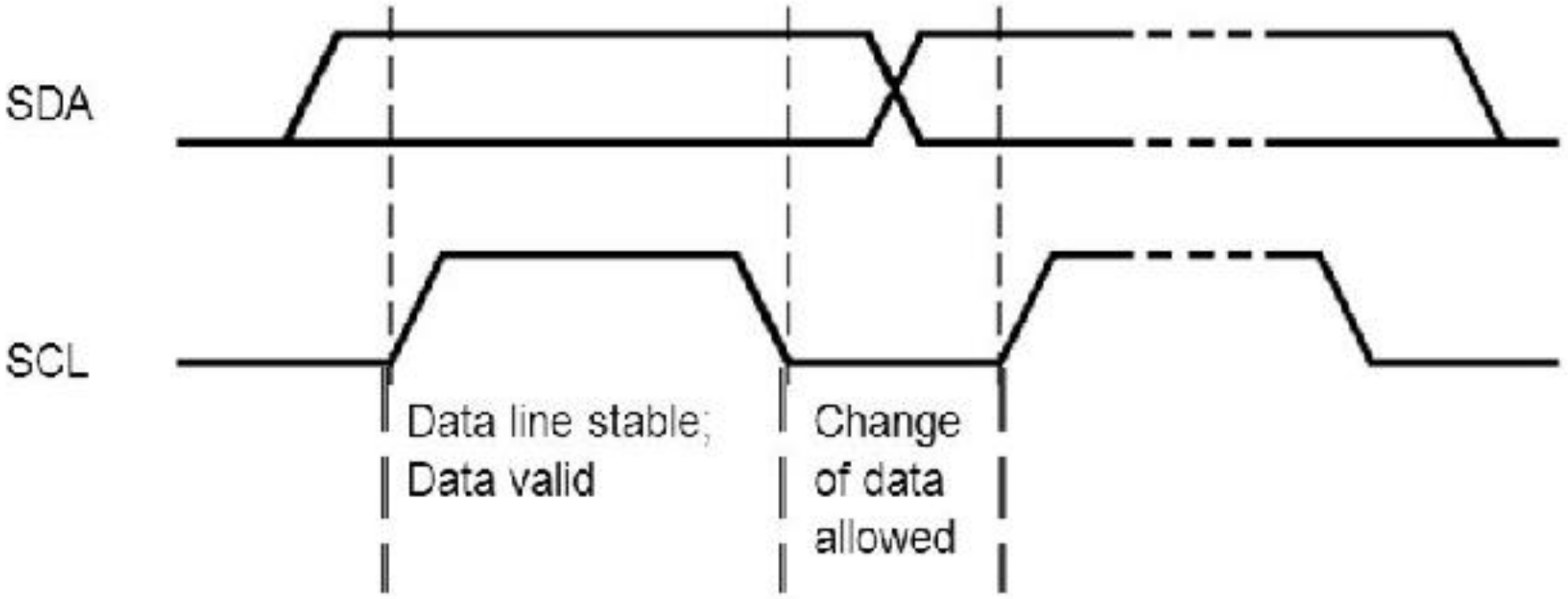
# I2c.c

```
void i2c_restart(void)
{
  // start condition
  I2CONSET=1<<STA_BIT;
  //clr SI_BIT
  I2CONCLR=1<<SI_BIT;
  //wait for SI bit status
while(((I2CONSET>>SI_BIT)&1)==0);
 // clear start condition
 I2CONCLR=1<<STA_BIT;
}
```

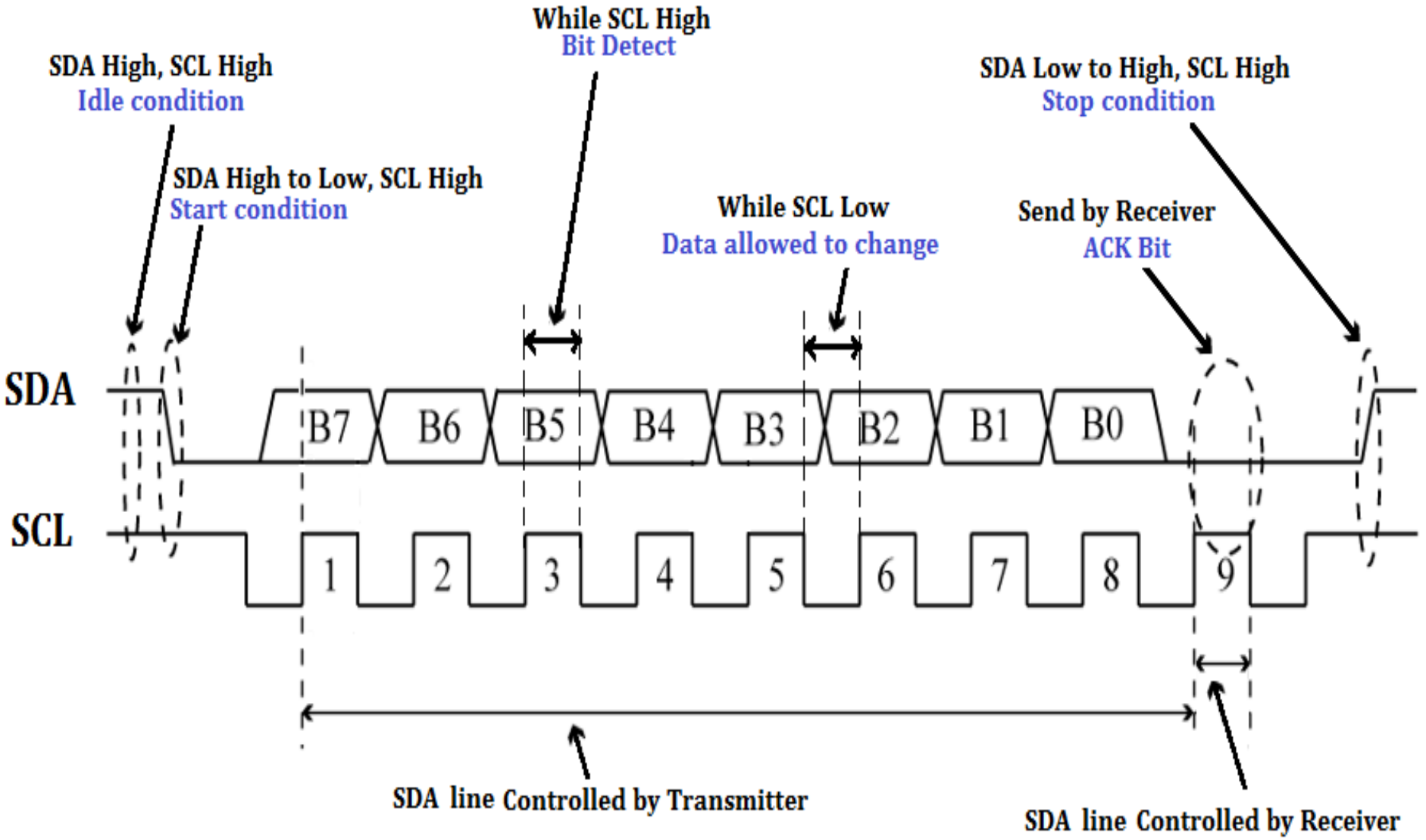
```
void i2cReStart(void)
{
SCLRBIT(IOCLR0,SCL);
SSETBIT(IOSET0,SDA);
SSETBIT(IOSET0,SCL);
SCLRBIT(IOCLR0,SDA);
}
```

# Bit Transfer on the I²C Bus

- In normal data transfer, the data line only changes state when the clock is low



# Data Validity & Transition



# I2c.c

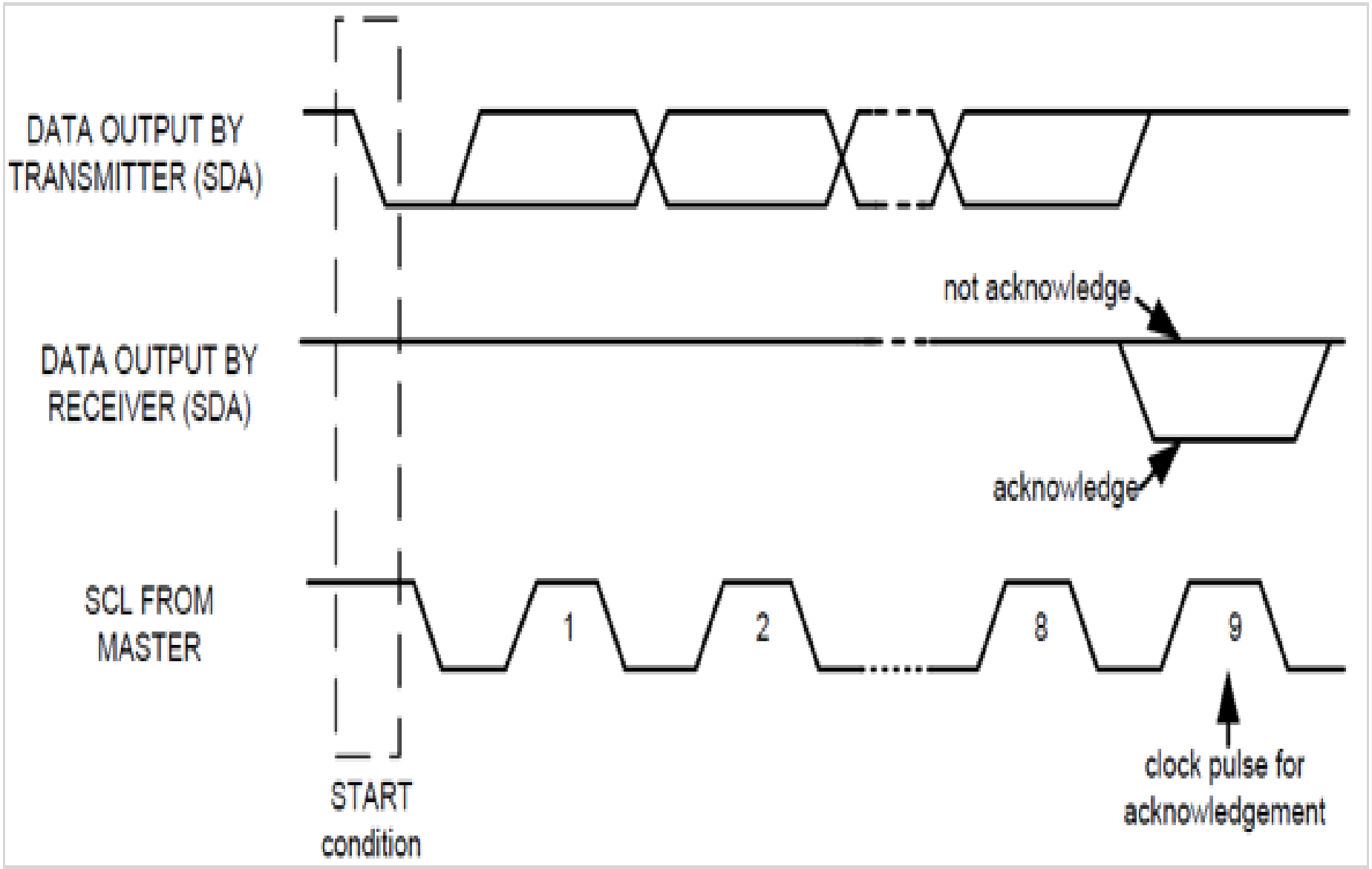
```
void i2c_write(u8 dat)
{
  //put data into I2DAT
  I2DAT=dat;
  //clr SI_BIT
  I2CONCLR = 1<<SI_BIT;
  //wait for SI bit status
 while(((I2CONSET>>SI_BIT)&1)==0);
}
```

```
void i2cWrite(u8 sDat)
{
   s32 i;
   for(i=7;i>=0;i--)
   {
     SCLRBIT(IOCLR0,SCL);
READWRITEBIT2(IOPIN0,SDA,sDat,i);
SSETBIT(IOSET0,SCL);
   }
}
```

# I2c.c

```
u8 i2cRead(void)
{
   s32 i;
   u8 buff;
   CLRBIT(IODIR0,SDA);
   for(i=7;i>=0;i--)
   {
     SCLRBIT(IOCLR0,SCL);//M  makes scl low so S can change sda
     delay_us(1);
     SSETBIT(IOSET0,SCL);//M  makes scl high to validate/read sda
    READWRITEBIT2(buff,i,IOPIN0,SDA);
   }
   SETBIT(IODIR0,SDA);
   return buff;
}
```

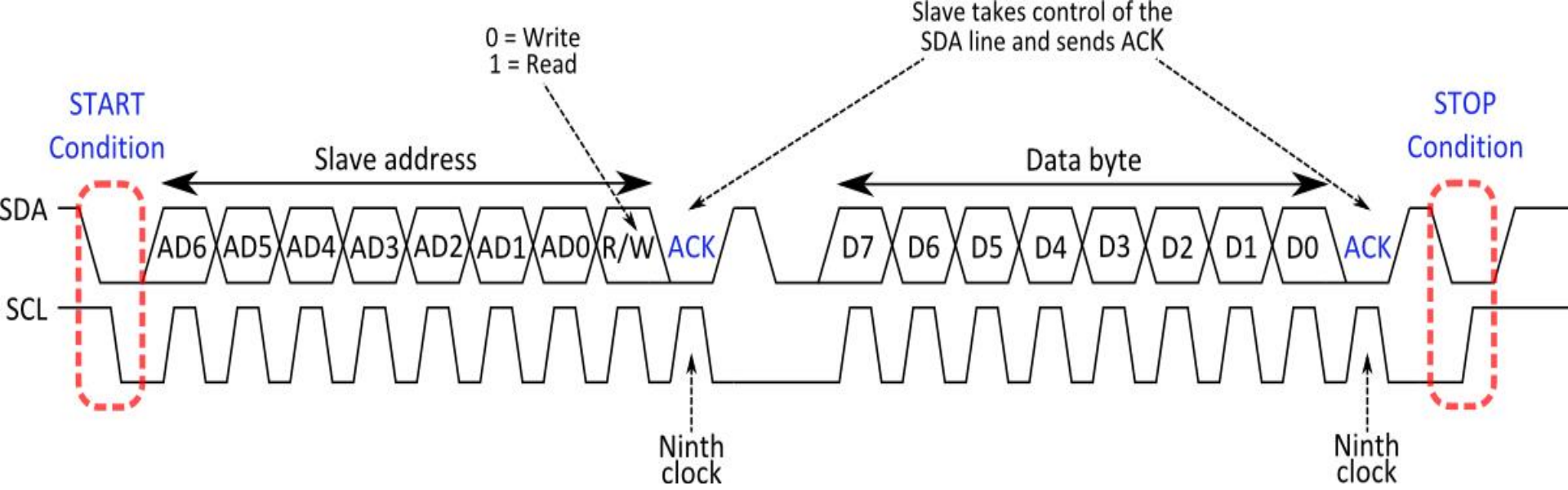
# ACK & NOT ACK(NACK)

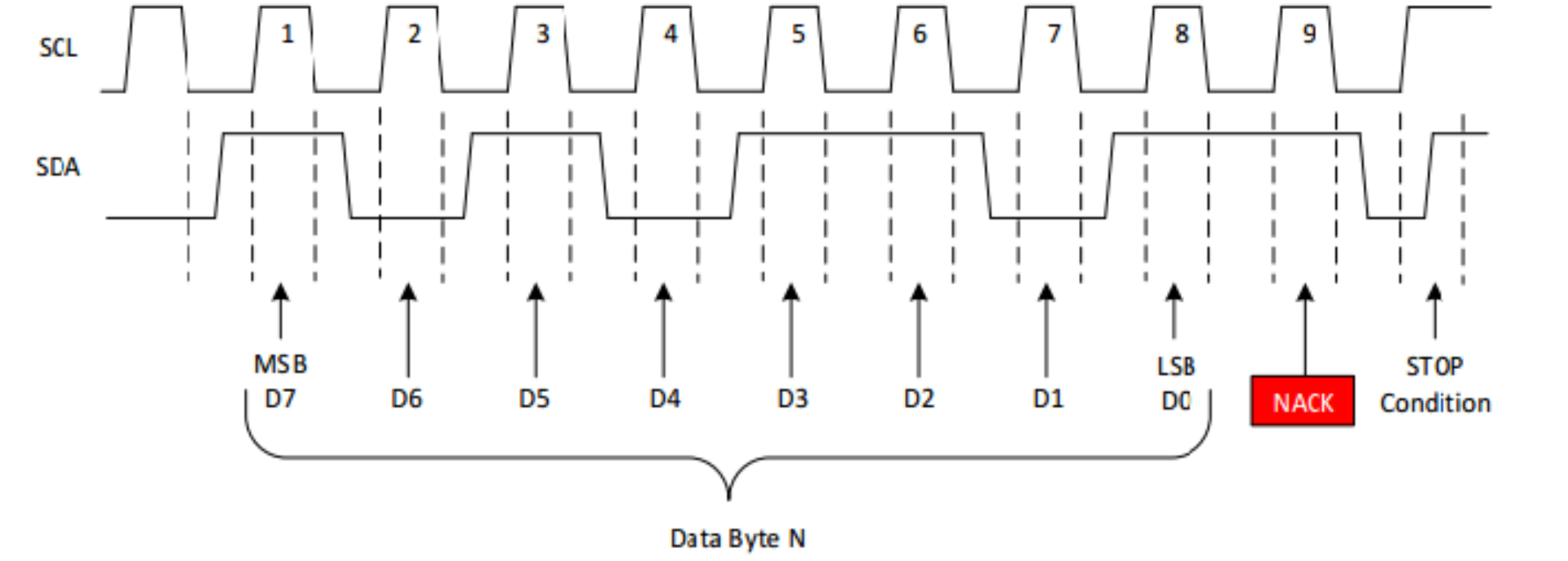


```
u8 i2c_nack(void)
{
  //Assert Not of Ack
     I2CONSET = 0x00;
    //Clear SI bit
    I2CONCLR = 1<<SI_BIT;
    //wait for byte reception
   while(((I2CONSET>>SI_BIT)&1)==0);
    //read received byte;
    return I2DAT;
}
```

```
void i2cAck(void)
{
    SCLRBIT(IOCLR0,SCL);
    SSETBIT(IOSET0,SDA);
    CLRBIT(IODIR0,SDA);
    SSETBIT(IOSET0,SCL);
    while(READBIT(IOPIN0,SDA)==1);
    SETBIT(IODIR0,SDA);
}

void i2cNAck(void)
{
   SCLRBIT(IOCLR0,SCL);
   SSETBIT(IOSET0,SDA);
   SSETBIT(IOSET0,SCL);
}
```


0 = Write
1 = Read
Slave takes control of the
SDA line and sends ACK
START
Condition
Slave address
Data byte
STOP
Condition
SDA
SCL
AD6
AD5
AD4
AD3
AD2
AD1
AD0
R/W
ACK
D7
D6
D5
D4
D3
D2
D1
D0
ACK
Ninth
clock
Ninth
clock


SCL
SDA
1
2
3
4
5
6
7
8
9
MSB
D7
D6
D5
D4
D3
D2
D1
LSB
D0
NACK
STOP
Condition
Data Byte N